

Constant Composition Distribution Matching

Patrick Schulte and Georg Böcherer, *Member, IEEE*

Abstract—Distribution matching transforms independent and Bernoulli(1/2) distributed input bits into a sequence of output symbols with a desired distribution. Fixed-to-fixed length, invertible, and low complexity encoders and decoders based on constant composition and arithmetic coding are presented. The encoder achieves the maximum rate, namely, the entropy of the desired distribution, asymptotically in the blocklength. Furthermore, the normalized divergence of the encoder output and the desired distribution goes to zero in the blocklength.

Index Terms—Distribution matching, fixed length, arithmetic coding, asymptotically optimal algorithm.

I. INTRODUCTION

A DISTRIBUTION MATCHER transforms independent Bernoulli($\frac{1}{2}$) distributed input bits into output symbols with a desired distribution. We measure the distance between the matcher output distribution and the desired distribution by normalized *informational divergence* [1, p. 7]. Informational divergence is also known as *Kullback-Leibler divergence* or *relative entropy* [2, Sec. 2.3]. A dematcher performs the inverse operation and recovers the input bits from the output symbols. A distribution matcher is a building block of the bootstrap scheme [3] that achieves the capacity of arbitrary discrete memoryless channels [4]. Distribution matchers are used in [5, Sec. VI] for rate adaptation and in [6] to achieve the capacity of the additive white Gaussian noise channel.

Prefix-free distribution matching was proposed in [7, Sec. IV.A]. In [8] and [9], Huffman codes are used for matching. Optimal variable-to-fixed and fixed-to-variable length distribution matchers are proposed in [10] and [11], respectively. The codebooks of the matchers in [8]–[11] must be generated offline and stored. This is infeasible for large codeword lengths, which are necessary to achieve the maximum rate. This problem is solved in [12] and [13] by using arithmetic coding to calculate the codebook online. The matchers proposed in [12] and [13] are asymptotically optimal. All approaches [8]–[13] are variable length, which can lead to varying transmission rate, large buffer sizes, error propagation and synchronization problems [8, Sec. I]. *Fixed-to-fixed* (f2f) length codes do not have these issues.

Manuscript received March 18, 2015; revised August 10, 2015; accepted September 18, 2015. Date of publication November 9, 2015; date of current version December 18, 2015. This work was supported by the German Federal Ministry of Education and Research in the framework of an Alexander von Humboldt Professorship.

The authors are with the Institute for Communications Engineering, Technische Universität München, Munich 80333, Germany (e-mail: patrick.schulte@tum.de; georg.boecherer@tum.de).

Communicated by E. Tuncel, Associate Editor for Source Coding.

Color versions of one or more of the figures in this paper are available online at <http://ieeexplore.ieee.org>.

Digital Object Identifier 10.1109/TIT.2015.2499181

The author of [14, Sec. 4.8] suggests to concatenate short codes and Mondelli *et al.* [4] employ a forward error correction decoder to build an f2f length matcher. The dematchers of [4] and [14] cannot always recover the input sequence with zero error. Hence systematic errors are introduced that cannot be corrected by the error correction code or by retransmission. The thesis [15] proposes an invertible f2f length distribution matcher called *adaptive arithmetic distribution matcher* (aadm). The algorithm is computationally complex.

In this work we propose practical, invertible, f2f length distribution matchers. They are asymptotically optimal and are based on constant composition codes indexed by arithmetic coding. The paper is organized as follows. In Section II we formally define distribution matching. We analyze constant composition codes in Section III. In Section IV we show how a constant composition distribution matcher (ccdm) and dematcher can be implemented efficiently by arithmetic coding.

II. PROBLEM STATEMENT

The entropy of a discrete random variable $\mathbf{A}$ with alphabet $\mathcal{A}$ and distribution $P_{\mathbf{A}}$ is

$$\mathbb{H}(\mathbf{A}) = \sum_{a \in \text{supp}(P_{\mathbf{A}})} -P_{\mathbf{A}}(a) \log_2 P_{\mathbf{A}}(a) \quad (1)$$

where $\text{supp}(P_{\mathbf{A}}) \subseteq \mathcal{A}$ is the support of $P_{\mathbf{A}}$. The informational divergence of two distributions on $\mathcal{A}$ is

$$\mathbb{D}(P_{\hat{\mathbf{A}}} \| P_{\mathbf{A}}) = \sum_{a \in \text{supp}(P_{\hat{\mathbf{A}}})} P_{\hat{\mathbf{A}}}(a) \log_2 \frac{P_{\hat{\mathbf{A}}}(a)}{P_{\mathbf{A}}(a)}. \quad (2)$$

The normalized informational divergence for length n random vectors $\hat{\mathbf{A}}^n = \hat{\mathbf{A}}_1 \dots \hat{\mathbf{A}}_n$ and $\mathbf{A}^n$ is defined as

$$\frac{\mathbb{D}(P_{\hat{\mathbf{A}}^n} \| P_{\mathbf{A}^n})}{n}. \quad (3)$$

For random vectors with independent and identically distributed (iid) entries, we write

$$P_{\mathbf{A}}^n(a^n) = \prod_{i=1}^n P_{\mathbf{A}}(a_i). \quad (4)$$

A one-to-one f2f distribution matcher is an invertible function f . We denote the inverse function by f^{-1} . The mapping imitates a desired distribution $P_{\mathbf{A}}$ by mapping m Bernoulli($\frac{1}{2}$) distributed bits $\mathbf{B}^m$ to length n strings $\hat{\mathbf{A}}^n = f(\mathbf{B}^m) \in \mathcal{A}^n$. The output distribution is $P_{\hat{\mathbf{A}}^n}$. The concept of one-to-one f2f distribution matching is illustrated in Fig. 1.

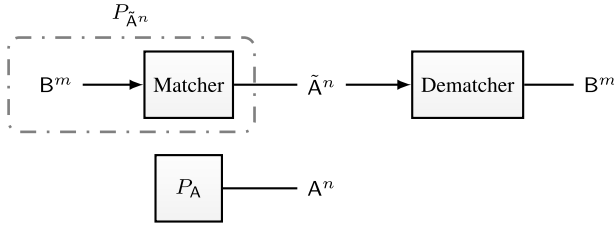


Fig. 1. Matching a data block $\mathbf{B}^m = \mathbf{B}_1 \dots \mathbf{B}_m$ to output symbols $\tilde{\mathbf{A}}^n = \tilde{\mathbf{A}}_1 \dots \tilde{\mathbf{A}}_n$ and reconstructing the original sequence at the dematcher. The rate is $\frac{m}{n} \left[\frac{\text{bits}}{\text{output symbol}} \right]$. The matcher can be interpreted as emulating a discrete memoryless source $P_{\tilde{\mathbf{A}}}$, i.e., $P_{\tilde{\mathbf{A}}}^n$ and $P_{\mathbf{A}}^n$ are close in informational divergence.

Definition 1: A matching rate $R = m/n$ is achievable for a distribution $P_{\mathbf{A}}$ if for any $\alpha > 0$ and sufficiently large n there is an invertible mapping $f: \{0, 1\}^m \rightarrow \mathcal{A}^n$ for which

$$\frac{\mathbb{D}(P_{f(\mathbf{B}^m)} || P_{\mathbf{A}}^n)}{n} \leq \alpha. \quad (5)$$

The following proposition in [16] relates the rate R and (5).

Proposition 1 (Converse, [16, Proposition 8]): There exists a positive-valued function β with

$$\beta(\alpha) \xrightarrow{\alpha \rightarrow 0} 0 \quad (6)$$

such that (5) implies

$$\frac{m}{n} \leq \frac{\mathbb{H}(\mathbf{A})}{\mathbb{H}(\mathbf{B})} + \beta(\alpha). \quad (7)$$

Proposition 1 bounds the maximum rate that can be achieved under condition (5). Since $\mathbb{H}(\mathbf{B}) = 1$ we have

$$R \leq \mathbb{H}(\mathbf{A}) \quad (8)$$

for any achievable rate R .

III. CONSTANT COMPOSITION DISTRIBUTION MATCHING

The empirical distribution of a vector $\mathbf{c}$ of length n is defined as

$$P_{\tilde{\mathbf{A}}, \mathbf{c}}(a) = \frac{n_a(\mathbf{c})}{n} \quad (9)$$

where $n_a(\mathbf{c}) = |\{i : c_i = a\}|$ is the number of times symbol a appears in $\mathbf{c}$. The authors of [17, Sec. 2.1] call $P_{\tilde{\mathbf{A}}, \mathbf{c}}$ the *type* of $\mathbf{c}$. An n -type is a type based on a length n sequence. A codebook $\mathcal{C}_{\text{ccdm}} \subseteq \mathcal{A}^n$ is called a *constant composition code* if all codewords are of the same type, i.e., $n_a(\mathbf{c})$ does not depend on the codeword $\mathbf{c}$. We will write n_a in place of $n_a(\mathbf{c})$ for a constant composition code.

A. Approach

We use a constant composition code with $n_a \approx P_{\mathbf{A}} \cdot n$. As all n_a need to be integers and add up to n , there are multiple possibilities to choose the n_a . We use the allocation that solves

$$\begin{aligned} P_{\tilde{\mathbf{A}}} &= \underset{P_{\tilde{\mathbf{A}}'}}{\operatorname{argmin}} \mathbb{D}(P_{\tilde{\mathbf{A}}} || P_{\mathbf{A}}) \\ &\text{subject to } P_{\tilde{\mathbf{A}}'} \text{ is } n\text{-type.} \end{aligned} \quad (10)$$

The solution of (10) can be found efficiently by [18, Algorithm 2]. This allocation provides a clear rule for choosing the n_a and is convenient for analysis. Suppose the output length n is fixed and that we can choose the input length m . Let $\mathcal{T}_{P_{\tilde{\mathbf{A}}}}^n$ be the set of vectors of type $P_{\tilde{\mathbf{A}}}$, i.e., we have

$$\mathcal{T}_{P_{\tilde{\mathbf{A}}}}^n = \left\{ \mathbf{v} \mid \mathbf{v} \in \mathcal{A}^n, \frac{n_a(\mathbf{v})}{n} = P_{\tilde{\mathbf{A}}}(a) \quad \forall a \in \mathcal{A} \right\}. \quad (11)$$

The matcher is invertible, so we need at least as many codewords as input blocks. The input blocklength must thus not exceed $\log_2 |\mathcal{T}_{P_{\tilde{\mathbf{A}}}}^n|$. We set the input length to $m = \lfloor \log_2 |\mathcal{T}_{P_{\tilde{\mathbf{A}}}}^n| \rfloor$ and we define the encoding function

$$f_{\text{ccdm}}: \{0, 1\}^m \rightarrow \mathcal{T}_{P_{\tilde{\mathbf{A}}}}^n. \quad (12)$$

The actual mapping f_{ccdm} can be implemented efficiently by arithmetic coding, as we will show in Section IV. The constant composition codebook is now given by the image of f_{ccdm} , i.e.,

$$\mathcal{C}_{\text{ccdm}} = f_{\text{ccdm}}(\{0, 1\}^m). \quad (13)$$

Since f_{ccdm} is invertible, the codebook size is $|\mathcal{C}_{\text{ccdm}}| = 2^m$.

B. Analysis

We show that f_{ccdm} asymptotically achieves all rates satisfying (8). We can bound m by

$$m = \lfloor \log_2 |\mathcal{T}_{P_{\tilde{\mathbf{A}}}}^n| \rfloor \geq \log_2 |\mathcal{T}_{P_{\tilde{\mathbf{A}}}}^n| - 1. \quad (14)$$

Recall that the matcher output distribution is $P_{\tilde{\mathbf{A}}}^n$. We have

$$\begin{aligned} \mathbb{D}(P_{\tilde{\mathbf{A}}}^n || P_{\mathbf{A}}^n) &= \sum_{a^n \in \mathcal{C}_{\text{ccdm}} \subseteq \mathcal{T}_{P_{\tilde{\mathbf{A}}}}^n} 2^{-m} \log_2 \frac{2^{-m}}{P_{\mathbf{A}}^n(a^n)} \frac{P_{\tilde{\mathbf{A}}}^n(a^n)}{P_{\tilde{\mathbf{A}}}^n(a^n)} \\ &= \mathbb{D}(P_{\tilde{\mathbf{A}}}^n || P_{\tilde{\mathbf{A}}}^n) + \sum_{a^n \in \mathcal{C}_{\text{ccdm}} \subseteq \mathcal{T}_{P_{\tilde{\mathbf{A}}}}^n} 2^{-m} \log_2 \frac{P_{\tilde{\mathbf{A}}}^n(a^n)}{P_{\tilde{\mathbf{A}}}^n(a^n)} \\ &= \mathbb{D}(P_{\tilde{\mathbf{A}}}^n || P_{\tilde{\mathbf{A}}}^n) + |\mathcal{C}_{\text{ccdm}}| 2^{-m} \sum_{a \in \mathcal{A}} n_a \log_2 \frac{P_{\tilde{\mathbf{A}}}(a)}{P_{\mathbf{A}}(a)} \\ &= \underbrace{\mathbb{D}(P_{\tilde{\mathbf{A}}}^n || P_{\tilde{\mathbf{A}}}^n)}_{\text{Term 1}} + \underbrace{n \mathbb{D}(P_{\tilde{\mathbf{A}}} || P_{\mathbf{A}})}_{\text{Term 2}}. \end{aligned} \quad (15)$$

For Term 1 we obtain

$$\begin{aligned} \mathbb{D}(P_{\tilde{\mathbf{A}}}^n || P_{\mathbf{A}}^n) &= \sum_{a^n \in \mathcal{C}_{\text{ccdm}} \subseteq \mathcal{T}_{P_{\tilde{\mathbf{A}}}}^n} 2^{-m} \log_2 \frac{2^{-m}}{\prod_{i \in \mathcal{A}} P_{\tilde{\mathbf{A}}}(i)^{n_i}} \\ &= \sum_{\mathcal{C}_{\text{ccdm}}} 2^{-m} \log_2 \frac{2^{-m}}{2^{-n \mathbb{H}(\tilde{\mathbf{A}})}} \\ &= n \mathbb{H}(\tilde{\mathbf{A}}) - m. \end{aligned} \quad (16)$$

Using (16) in (15) and dividing by n we have

$$\frac{\mathbb{D}(P_{\tilde{\mathbf{A}}}^n || P_{\mathbf{A}}^n)}{n} = \mathbb{H}(\tilde{\mathbf{A}}) - R + \mathbb{D}(P_{\tilde{\mathbf{A}}} || P_{\mathbf{A}}). \quad (17)$$

The choice (10) of $P_{\tilde{\mathbf{A}}}$ minimizes the third term on the right-hand side of (17) and guarantees (see [18, Proposition 4])

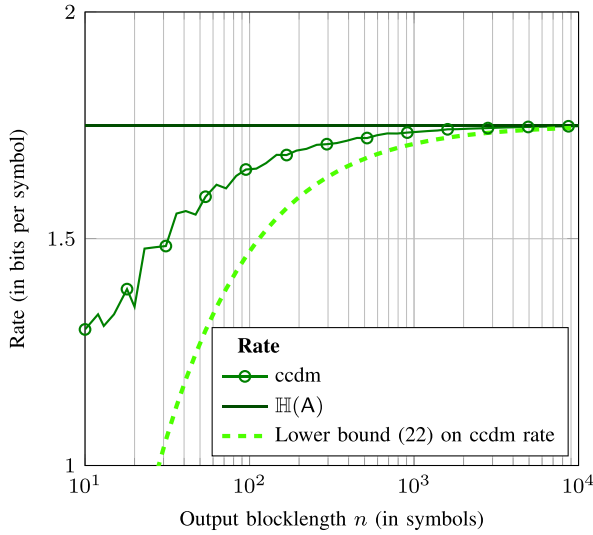


Fig. 2. Rates of ccdm versus output blocklength for $P_A = (0.0722, 0.1654, 0.3209, 0.4415)$.

that

$$\mathbb{D}(P_{\bar{A}}||P_A) < \log_2 \left(1 + \frac{k}{\min_{a \in \text{supp } P_A} P_A(a)n^2} \right) \quad (18)$$

where $k = |\mathcal{A}|$ is the alphabet size. Consequently, we know that this term vanishes as the blocklength approaches infinity, i.e., we have

$$\lim_{n \rightarrow \infty} \mathbb{D}(P_{\bar{A}}||P_A) = 0. \quad (19)$$

We now relate the input and output lengths to understand the asymptotic behavior of the rate. By [17, Lemma 2.2], we have

$$|\mathcal{T}_{P_A}^n| \geq \binom{n+k-1}{k-1} 2^{n\mathbb{H}(\bar{A})} \geq (n+k-1)^{-(k-1)} 2^{n\mathbb{H}(\bar{A})}. \quad (20)$$

Taking the logarithm to the base 2 and dividing by n we have

$$\frac{\log_2 |\mathcal{T}_{P_A}^n|}{n} \geq \frac{-(k-1) \log_2(n+k-1)}{n} + \mathbb{H}(\bar{A}). \quad (21)$$

For the rate, we obtain

$$\begin{aligned} R &= \frac{m}{n} \stackrel{(14)}{\geq} \frac{\log_2 |\mathcal{T}_{P_A}^n|}{n} - \frac{1}{n} \\ &\stackrel{(21)}{\geq} \frac{-(k-1) \log_2(n+k-1)}{n} + \mathbb{H}(\bar{A}) - \frac{1}{n} \end{aligned} \quad (22)$$

and in the asymptotic case

$$\lim_{n \rightarrow \infty} R = \mathbb{H}(\bar{A}). \quad (23)$$

From (19) and [16, Proposition 6] we know that $\mathbb{H}(\bar{A}) \rightarrow \mathbb{H}(A)$ and by (19) and (23) in (17), normalized divergence approaches zero for $n \rightarrow \infty$.

Example 1: The desired distribution is

$$P_A = (0.0722, 0.1654, 0.3209, 0.4415).$$

Fig. 2 and Fig. 3 show the rates and normalized divergences of ccdm and the optimal f2f length

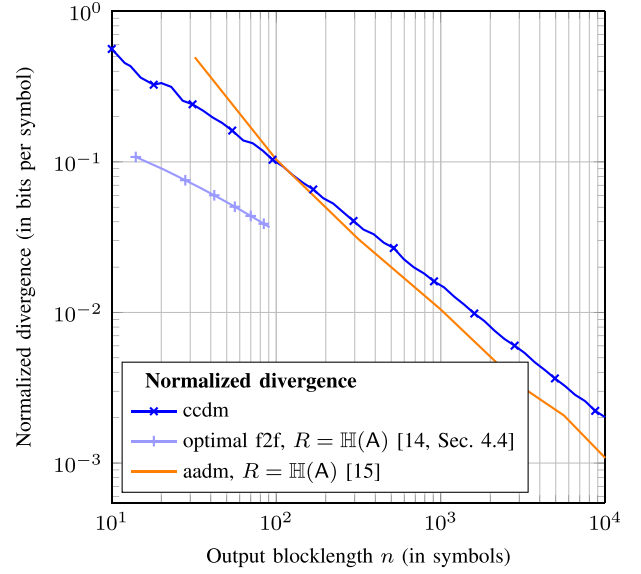


Fig. 3. Normalized divergence of ccdm versus output blocklength for $P_A = (0.0722, 0.1654, 0.3209, 0.4415)$. For comparison, the performance of optimal f2f [14, Sec. 4.4] and aadm [15] is displayed. Because of limited computational resources, we could calculate the performance of optimal f2f only up to a blocklength of $n = 90$.

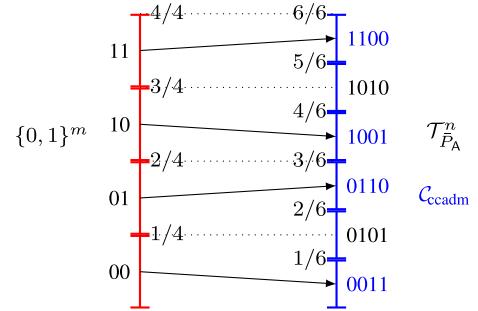


Fig. 4. Diagram of a constant composition arithmetic encoder with $P_{\bar{A}}(0) = P_{\bar{A}}(1) = 0.5$, $m = 2$ and $n = 4$.

matcher [14, Sec. 4.4], respectively. The empirical performance of aadm [15] is also displayed. For optimal f2f and aadm, the rate is fixed to $\mathbb{H}(A)$ bits per symbol. Fig. 3 shows that the ccdm needs about 160 symbols to reach an informational divergence of 0.06 bits per symbol, which is about 4 times the blocklength of the optimal scheme. However, the memory for storing the optimal codebook grows exponentially in m as there are 2^m codewords. In this example, ccdm performs better than aadm for short blocklength up to 100 symbols. Fig. 2 also shows the lower and upper bounds (8) and (22), respectively.

IV. ARITHMETIC CODING

We use arithmetic coding for indexing sequences efficiently. In [19] an m -out-of- n coding scheme was introduced that uses the same technique.

Our arithmetic encoder associates an interval to each input sequence in $\{0,1\}^m$ and it associates an interval to each output sequence in $\mathcal{T}_{P_A}^n$, see Fig. 4 for an example. The size of an interval is equal to the probability of the corresponding sequence according to the input and output model, respectively. For the input model we choose an iid

Bernoulli($\frac{1}{2}$) process. We describe the output model by a random vector

$$\bar{A}^n = \bar{A}_1 \bar{A}_2 \dots \bar{A}_n \quad (24)$$

with marginals $P_{\bar{A}_i} = P_{\bar{A}}$ and the uniform distribution

$$P_{\bar{A}^n}(a^n) = \frac{1}{|T_{P_{\bar{A}}}^n|} \quad \forall a^n \in T_{P_{\bar{A}}}^n.$$

The intervals are ordered lexicographically. All input and output intervals range from 0 to 1 because all probabilities add up to 1.

Example 2: Fig. 4 shows input and output intervals with output length $n = 4$ and $P_{\bar{A}}(0) = P_{\bar{A}}(1) = 0.5$. There are 4 equally probable input sequences and 6 equally probable output sequences. The intervals on the input side are $[0, 0.25]$, $[0.25, 0.5]$, $[0.5, 0.75]$ and $[0.75, 1]$. The intervals on the output side are $[0, \frac{1}{6}]$, $[\frac{1}{6}, \frac{2}{6}]$, $[\frac{2}{6}, \frac{3}{6}]$, $[\frac{3}{6}, \frac{4}{6}]$, $[\frac{4}{6}, \frac{5}{6}]$ and $[\frac{5}{6}, 1]$.¹

The arithmetic encoder can link an output sequence to an input sequence if the lower border of the output interval is inside the input interval. In the example (Fig. 4) ‘00’ may link to both ‘0101’ and ‘0011’, while for ‘01’ only a link to ‘0110’ is possible. There are at most two possible choices because by (14) the input interval size is less than twice the output interval size. Both choices are valid and we can perform an inverse operation. In our implementation, the encoder decides for the output sequence with the lowest interval border. As a result, the codebook C_{ccdm} of Example 2 is $\{‘0011’, ‘0110’, ‘1001’, ‘1100’\}$. In general C_{ccdm} has cardinality 2^m with $2^m \leq |T_{P_{\bar{A}}}^n| < 2^{m+1}$ according to (14). It is not possible to index the whole set $T_{P_{\bar{A}}}^n$ unless $2^m = |T_{P_{\bar{A}}}^n|$. The analysis of the code (Section III-B) is valid for all codebooks $C_{\text{ccdm}} \subseteq T_{P_{\bar{A}}}^n$. The actual subset is implicitly defined by the arithmetic encoder.

We now discuss the online algorithm that processes the input sequentially. Initially, the input interval spans from 0 to 1. As the input model is Bernoulli($\frac{1}{2}$) we split the interval into two equally sized intervals and continue with the upper interval in case the first input bit is ‘1’; otherwise we continue with the lower interval. After the next input bit arrives we repeat the last step. After m input bits we reach a size 2^{-m} interval. After every refinement of the input interval the algorithm checks for a sure prefix of the output sequence, e.g., in Fig. 4 we see that if the input starts with 1 the output must start with 1. Every time we extend the sure prefix by a new symbol, we must calculate the probability of the next symbol given the sure prefix. That means we determine the output intervals within the sure interval of the prefix. The model for calculating the conditioned probabilities is based on drawing without replacement. There is a bag with n symbols of k discriminable kinds. n_a denotes how many symbols of kind a are initially in the bag and n'_a is the current number. The probability to draw a symbol of type a is n'_a/n . If we pick a symbol a both n and n'_a decrement by 1.

Example 3: Fig. 5 shows a refinement of the output intervals. Initially there are 2 ‘0’s and 2 ‘1’s in the bag.

¹Please note that in this case no distribution matcher is needed. However, this indexing problem is of interest in its own right.

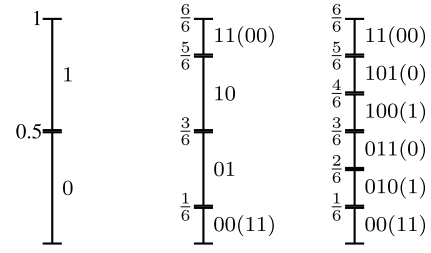


Fig. 5. Refinement of the output intervals. Round brackets indicate symbols that must follow with probability one.

The distribution of the first drawn symbol is $P_{\bar{A}_1}(0) = P_{\bar{A}_1}(1) = \frac{1}{2}$. When drawing a ‘0’, there are 3 symbols remaining: one ‘0’ and two ‘1’s. Thus, the probability for a ‘0’ reduces to $1/3$ while the probability of ‘1’ is $2/3$. If two ‘0’s were picked, two ‘1’s must follow. This way we ensure that the encoder output is of the desired type. Observe that the probabilities of the next symbol depend on the previous symbols, e.g., we have

$$P_{\bar{A}_2|\bar{A}_1}(0|0) \neq P_{\bar{A}_2|\bar{A}_1}(0|1) \quad (25)$$

in general. However, $P_{\bar{A}^n}(a^n) = \prod_{i=1}^n P_{\bar{A}_i|\bar{A}^{i-1}}(a_i|a^{i-1})$ is constant on $T_{P_{\bar{A}}}^n$ as we show in the following proposition.

Proposition 2: After n refinements of the output interval the model used for the refinement step stated above creates equally spaced (equally probable) intervals that are labeled with all sequences in $T_{P_{\bar{A}}}^n$.

Proof: All symbols in the bag are chosen at some point. Consequently only sequences in $T_{P_{\bar{A}}}^n$ may appear. All possibilities associated with the chosen string are products of fractions n'_a/n , where n takes on all values from the initial value to 1 because every symbol is drawn at some point. Thus for each string we obtain for its probability an expression that is independent of the realization itself:

$$P_{\bar{A}^n}(a^n) = \frac{n_{a=0}! \dots n_{a=k-1}!}{n!} = \frac{1}{|T_{P_{\bar{A}}}^n|} \quad \forall a^n \in T_{P_{\bar{A}}}^n. \quad (26)$$

□

Numerical problems for representing the input interval and the output interval occur after a certain number of input bits. For this reason we introduce a *rescaling* each time a new output symbol is known. We explain this next.

A. Scaling Input and Output Intervals

After we identify a sure prefix, we are no longer interested in code sequences that do not have that prefix. We scale the input and output interval such that the output interval is $[0, 1]$. Fig. 6 illustrates the mapping of intervals (in_1, out_1) to (in_2, out_2) . The refinement for the second symbol works as described in Example 3. If the second input bit is 0, we know that 10 must be a prefix of the output. The resulting scaling is shown in Fig. 6 as (in_2, out_2) to (in_3, out_3) . A more detailed explanation of scaling for arithmetic coding can be found for instance in [20, Ch. 4]. We provide an implementation of ccdm online [21].

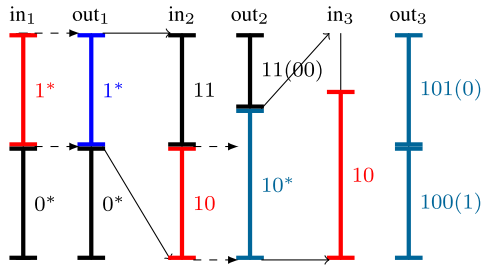


Fig. 6. Scaling of input and output intervals in case the input interval is a subset of an output interval. The latter interval corresponds to $[0, 1)$ after scaling. A star indicates that this is just a prefix of the complete word. Round brackets indicate symbols that must follow with probability one.

V. CONCLUSION

We presented a practical and invertible f2f length distribution matcher that achieves the maximum rate asymptotically in the blocklength. In contrast to matchers proposed in the literature [8]–[13] the f2f matcher is robust to synchronization and variable rate problems. In future work we plan to investigate f2f length codes that perform well in the finite blocklength regime.

ACKNOWLEDGMENT

The authors wish to thank Irina Bocharova and Boris Kudryashov for encouraging them to work on the presented approach.

REFERENCES

- [1] I. Csiszár and J. Körner, *Information Theory: Coding Theorems for Discrete Memoryless Systems*. Cambridge, U.K.: Cambridge Univ. Press, 2011.
- [2] T. M. Cover and J. A. Thomas, *Elements of Information Theory*, 2nd ed. New York, NY, USA: Wiley, 2006.
- [3] G. Böcherer and R. Mathar, "Operating LDPC codes with zero shaping gap," in *Proc. IEEE Inf. Theory Workshop (ITW)*, Paraty, Brazil, Oct. 2011, pp. 330–334.
- [4] M. Mondelli, R. Urbanke, and S. H. Hassani, "How to achieve the capacity of asymmetric channels," in *Proc. 52nd Annu. Allerton Conf. Commun., Control, Comput.*, Monticello, IL, USA, Sep./Oct. 2014, pp. 789–796.
- [5] D. J. C. MacKay, "Good error-correcting codes based on very sparse matrices," *IEEE Trans. Inf. Theory*, vol. 45, no. 2, pp. 399–431, Mar. 1999.
- [6] G. Böcherer, F. Steiner, and P. Schulte, "Bandwidth efficient and rate-matched low-density parity-check coded modulation," *IEEE Trans. Commun.*, vol. 63, no. 12, Dec. 2015.
- [7] J. G. Forney, R. G. Gallager, G. Lang, F. M. Longstaff, and S. U. Qureshi, "Efficient modulation for band-limited channels," *IEEE J. Sel. Areas Commun.*, vol. 2, no. 5, pp. 632–647, Sep. 1984.
- [8] F. R. Kschischang and S. Pasupathy, "Optimal nonuniform signaling for Gaussian channels," *IEEE Trans. Inf. Theory*, vol. 39, no. 3, pp. 913–929, May 1993.
- [9] G. Ungerboeck, "Huffman shaping," in *Codes, Graphs, and Systems*, R. E. Blahut and R. Koetter, Eds. New York, NY, USA: Springer-Verlag, 2002, ch. 17, pp. 299–313.
- [10] G. Böcherer and R. Mathar, "Matching dyadic distributions to channels," in *Proc. Data Compression Conf.*, Snowbird, UT, USA, Mar. 2011, pp. 23–32.
- [11] R. A. Amjad and G. Böcherer, "Fixed-to-variable length distribution matching," in *Proc. IEEE Int. Symp. Inf. Theory (ISIT)*, Istanbul, Turkey, Jul. 2013, pp. 1511–1515.
- [12] N. Cai, S.-W. Ho, and R. W. Yeung, "Probabilistic capacity and optimal coding for asynchronous channel," in *Proc. IEEE Inf. Theory Workshop (ITW)*, Lake Tahoe, CA, USA, Sep. 2007, pp. 54–59.
- [13] S. Baur and G. Böcherer, "Arithmetic distribution matching," in *Proc. 10th Int. ITG Conf. Syst., Commun., Coding*, Hamburg, Germany, Feb. 2015, pp. 1–6.
- [14] R. A. Amjad, "Algorithms for simulation of discrete memoryless sources," M.S. thesis, Inst. Commun. Eng., Technische Universität München, Munich, Germany, 2013.
- [15] P. Schulte, "Zero error fixed length distribution matching," M.S. thesis, Inst. Commun. Eng., Technische Universität München, Munich, Germany, 2014.
- [16] G. Böcherer and R. A. Amjad, "Informational divergence and entropy rate on rooted trees with probabilities," in *Proc. IEEE Int. Symp. Inf. Theory (ISIT)*, Honolulu, HI, USA, Jun./Jul. 2014, pp. 176–180.
- [17] I. Csiszár and P. C. Shields, "Information theory and statistics: A tutorial," *Found. Trends Commun. Inf. Theory*, vol. 1, no. 4, pp. 417–528, 2004.
- [18] G. Böcherer and B. C. Geiger, (Mar. 2015). "Optimal quantization for distribution synthesis." [Online]. Available: <http://arxiv.org/abs/1307.6843>
- [19] T. V. Ramabadran, "A coding scheme for m -out-of- n codes," *IEEE Trans. Commun.*, vol. 38, no. 8, pp. 1156–1163, Aug. 1990.
- [20] K. Sayood, *Introduction to Data Compression*. Amsterdam, The Netherlands: Elsevier, 2006.
- [21] *A Fixed-to-Fixed Length Distribution Matcher in C/MATLAB*. [Online]. Available: <http://beam.to/ccdm>, accessed Nov. 12, 2015.

Patrick Schulte received the B.S. and M.S. degree in Electrical Engineering both from Technische Universität München, Germany in 2012 and 2014, respectively. Since 2014, he is a doctoral student at the Institute of Communications Engineering of Technische Universität München. His current research interests are iterative channel coding and probabilistic shaping.

Georg Böcherer (S'05–M'14) was born in Freiburg im Breisgau, Germany. He obtained his M.Sc. degree in Electrical Engineering and Information Technology from the ETH Zürich in 2007, and his Ph.D. degree from the RWTH Aachen University in 2012. He is now a senior researcher at the Institute for Communications Engineering, Technische Universität München. His current research interests are coding, modulation, and probabilistic shaping for optical, wireless, and wired communications.